The Maze Runner

Search Algorithms

Group Number: 03

Member Name	Student ID
Aadil Hussain	BSCS25121
Muhammad Abdullah	BSCS25017

1. AI assistant used

We used OpenAI ChatGPT and Anthropic Claude as tools, for study and debugging. They helped us understand the supplied codebase learn how the search algorithms operate, spot edge cases and arrange and format this report. All code was written by us. We can explain every part of its implementation.

2. Results

The comparison limit is 10 seconds per algorithm; times are single-run measurements rounded by the runner.

maze_medium

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	162	162	233	0.008
BFS	66	66	407	0.003
IDS	66	66	437,730	7.190
UCS	66	66	407	0.006
Greedy	80	80	81	0.001
A*	66	66	209	0.003

open_glade

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	234	234	322	0.017
BFS	32	32	344	0.004
IDS (timeout)	-	-	430,423	10.001
UCS	32	32	344	0.021
Greedy	36	36	39	0.001
A*	32	32	114	0.005

ivy_maze

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	159	88	422	0.021
BFS	80	60	439	0.008
IDS (timeout)	-	-	432,311	10.000
UCS	70	64	418	0.024
Greedy	130	82	85	0.002
A*	70	64	344	0.010

3. DFS versus BFS

On maze_medium depth-first search finds a 162-step path while expanding 233 nodes. Breadth- search finds a shorter 66-step path but expands 407 nodes. Depth-first search explores deeply using a LIFO stack following the available direction first. It accepts the goal reached. An explored set prevents repeated expansions but not long detours. Both algorithms are complete on mazes, with sufficient resources.. Depth-first search may fail on infinite branches. Breadth- search guarantees the shortest path and minimum cost when all moves have equal positive costs. Depth-first search has no optimality guarantee. Breadth- search may not minimize cost when move costs differ.

4. Iterative deepening search (IDS)

I noticed that on maze_small IDS finds the 50-step solution in 0.040 seconds, with 1,352 expansions while on open_glade IDS reaches 430,423 expansions and times out after 10.001 seconds because many possible routes and repeated exploration. DLS stops cycles along the path but cells can still be revisited via other routes and IDS repeats this work as depth limits grow. A global visited set might wrongly cut off paths that need more depth. IDS uses memory than BFS because IDS avoids the very large frontier that BFS builds yet IDS keeps the shortest-path guarantee for equal costs. Still copying path sets and action lists can need a lot of storage about d squared, along a branch and this implementation limits depth attempts to 9,999.

5. Path costs: BFS versus UCS

I see that on griever_alley BFS finds a 20-step path costing 56. UCS finds a 28-step path costing 42 by avoiding expensive Griever cells and going through ivy. BFS works to reduce steps while UCS works to reduce terrain cost. Both give the total cost when every move costs the same positive amount even though the routes can be different. When terrain costs vary the results can be different as ivy_maze shows, with BFS costing 80 and UCS costing 70.

6. Greedy search and heuristics

On greed_trap Greedy finds a 25-step path costing 52. Expands 60 nodes while UCS and Manhattan A* find 25-step paths costing 25. Greedy prioritises estimated distance h without considering accumulated cost g which leads Greedy through three Griever cells. Euclidean distance is admissible and consistent because movement is cardinal and each move costs least 1; walls and expensive terrain only increase the true remaining cost. However an admissible heuristic does not guarantee optimality because it ignores the cost already paid.

A* on maze_big	Cost	Steps	Expanded	Time (s)
Manhattan	124	124	679	0.008
Euclidean	124	124	880	0.011

Manhattan expands 679 nodes versus Euclidean's 880: 201 fewer (22.8% fewer relative to Euclidean). Both return optimal cost 124. Manhattan is a tighter lower bound for four-direction movement and dominates Euclidean, so it gives A* stronger guidance in this run. The exact counts also reflect walls, ties and the fixed successor order.

7. Q9: key-hunting heuristic

The state is made up of the position and the keys that are still left stored as a frozenset. The heuristic checks every order of the keys adds up the shortest distances that know about walls between the position the keys and the exit and picks the smallest sum. When there are no keys left the heuristic simply uses the distance to the exit. Cached distances and states make the process faster. The heuristic is admissible because the heuristic finds the best cost for a simplified problem where each step costs one and that cost cannot be higher than the real cost, on the terrain. The heuristic is also consistent because every legal move costs least one so the inequality

$$h(s) \leq c(s, s') + h(s')$$

holds and the heuristic equals zero when we reach the goal. Consistency explains why A* does not reopen states that have already been expanded.

Key maze	A* cost	Steps	A* expanded	UCS expanded
keys_tiny	26	26	59	490
keys_small	54	54	91	1,723
keys_medium	94	94	117	9,695

Key runs used –compare -p keys (default heuristic: keyhunt). The maze_big runs used –compare -H manhattan and –compare -H euclidean; other examples used –compare. The Q9 expansion counts exclude the internal BFS work used to compute distances. Trying k! key orders is costly as the number of keys grows, despite caching. Full command output is supplied separately in comparison_results.txt.